

Applied Cloud Computing

Assignment 2 Amazon EC2 RDS (updated Sept. 2025)

In this assignment, you will create RDS(Relational Data Storage) instance and an EC2 instance that runs the http server that render html page with data from RDS . Here are the steps:

Step 1. Create Two Security Groups (named by sgRDS and sgWEB) within the same VPC(default VPC e.g.) , will be used by RDS instance and EC2 instance respectively.

Step 1.1, First create security group with name **sgWEB**, with description like “security group for Web access “. At the **Inbound rules**, select Type as **HTTP**(at port 80), Add 2nd rules **SSH**(at port 22). Both **Source** select **Anywhere-IPv4**. For **Outbound rules**, leave it as-is (All traffic). Click “Create security group”.

Step 1.2. Create 2nd security group with name **sgRDS**, with description, like “security group for RDS”. At the **Inbound rules**, select Type as **MYSQL/Aurora** (at port 3306). For **Source** location use the “find” icon and type in sgWEB or copy the sgWEB Security Group ID sg-xxxx into here. For **Outbound rules**, leave it as-is (All traffic). Click “Create security group”.

Now you should have two security groups (sgWEB and sgRDS) created.
More general info. is here:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Step 2. Create RDS instance

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateInstance.html

Note1: Free tier access include the following:

Amazon RDS is a managed relational database service for MySQL, PostgreSQL, MariaDB, Oracle BYOL, or SQL Server

12 MONTHS FREE : 750 hours per month of db.t2.micro, db.t3.micro, and db.t4g.micro Single-AZ instance usage (applicable database engines)

20 GB of general purpose (SSD) database storage

20 GB of storage for database backups and database snapshots

<https://aws.amazon.com/free/database/?p=ft&z=subnav&loc=3>

Note2: Set up a Budget Plan:

It is a good idea to set up a “Zero-Spend Budget” plan to track your spending to avoid unnecessary cost. After login, click on your user name at the upper right corner, then select “Billing and Cost Management”, In the ‘Choose budget type’, select template “Zero spend budget”, fill in your Email address, then click ‘Create budget’. This will create an Email alert which will notify you once your spending exceeds \$0.01 above the AWS Free Tier limits.

Go to <https://console.aws.amazon.com/rds/>

- In the upper-right corner of the AWS Management Console, check the AWS Region. It should be the same as the one where you created your EC2 instance. (for example, N. California). If it is different, change it to the same region.
- On the left hand menu, click “Databases”, then on the right side click “Create database”
- choose engine option – MySQL (the default is “Aurora(MySQL compatible)”, but **DO NOT pick that one**).
- use the default version
- Templates - “Sandbox”
- In Setting, DB cluster identifier - “database-cloudcompute” (or your favor)
- Credential setting - create admin password yourself (remember the password you set here)
- Instance Configuration – **Change to “db.t3.micro”**
- Cluster storage configuration keep as-is (General Purpose SSD(gp2)). Allocate storage change to 10 G.
- Connectivity
 - o In the Computer Resource, select “Don’t connect to an EC2 compute resource”
 - o Network Type, select “IPv4”
 - o In Virtual private cloud (VPC), keep things as-is.
 - o In Public Access, keep “No”.
 - o In the VPC security groups, select “Choose Existing” then use the pull down menu in the Existing VPC security group, select “sgRDS”
 - o Expand ‘Additional configuration’, make sure database port is ‘3306’
- Finally click to create the RDS database. This might take several minutes. Put down the RDS instance Endpoint (e.g. something like ‘database-cloudcompute.czeqmc60yl0u.us-west-1.rds.amazonaws.com’)

Step 3. Create EC2 instance:

As what we did in assignment 1 Part2 (via your own AMI image). When configure the instance, in the Network Setting -> Firewall (security group), click “Select Existing Security Group”, then use the pull down and select security group **sgWEB**. Click “Launch instance”. Now you have an EC2 instance up.

Step 4. Install MySQL client into you EC2 instance.

Step 4.1. ssh to the **EC2 instance** and install **mysql client**

```
$ sudo yum install -y https://dev.mysql.com/get/mysql57-community-release-el7-11.noarch.rpm
```

```
$ sudo yum install -y mysql-community-client
```

Note: you may get an error message like “Error: GPG check FAILED”. If so, run the next two instructions:

```
$ sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2022
```

```
$ sudo yum install -y mysql-community-client
```

Now you should have a successful mysql client installed.

Step 5. Create database:

First transfer the file 'createDB.sql' and 'test.py' from your **local file system** to EC2 instance via 'scp' command.

Note 1: **Make sure you are in your local system**(Mac or Windows)'s terminal(or putty window) before you try to execute the 'scp' command.

Note 2: example of scp command :

- `scp -i SCU_2025Fall.pem test.pdf ec2-user@ec2-54-193-166-215.us-west-1.compute.amazonaws.com:test.pdf`

After the above scp command successfully executed, now go to your **EC2 instance** and check whether those two files had been transferred correctly. If those two files are there, then you can proceed to next step: create the database in your RDS then test it.

In the RDS instance, you need to copy the "endpoint" first, something like "database-cloudcompute.czeqmc60.us-west-1.rds.amazonaws.com" , then execute the following command.

\$ mysql -h database-cloudcompute.czeqmc60.us-west-1.rds.amazonaws.com -P 3306 -u admin -p

(then input the admin password you created in previous steps to connect to the database, after successfully connect to mysql database, you will see the "mysql" prompt, issue the following commands:)

```
mysql> source createDB.sql;
mysql > show databases;
mysql > use testdb;
mysql > show tables;
mysql > select * from titanic;
mysql > \q;
```

Step 6. Test the database:

Now that you've connected to the RDS through EC2 and create database and table and insert some sample data, try to run the test.py from your EC2 instance to retrieve the data from the database

- you should already transferred the 'test.py' file from your local system to your EC2 instance, if not, use 'scp' to transfer the file.
- In the test.py file, **modify** the database related settings (db_host, db_pass) according to your RDS instance.
- Install the python3 and pymysql into your **EC2 instance** via
 - ***\$ sudo yum install python3***
 - ***\$ sudo python3 -m pip install PyMySQL***
- then you should be able to execute the 'test.py' to retrieve the data in the 'titanic' table in the 'testdb' database in RDS via:

- *\$ python3 test.py*

(below is the expected output from the titanic data)

uid:892, pclass:3, name:Kelly, sex:male, age:34.5

uid:893, pclass:3, name:Wilkes, sex:female, age:47.0

uid:894, pclass:2, name:Myles, sex:male, age:62.0

uid:900, pclass:3, name:Abraham, sex:female, age:18.0

uid:903, pclass:1, name:Jones, sex:male, age:46.0

Step 7. Display the Titanic table in graphic mode in the browser:

If everything succeeds, now you need to integrate the retrieve function in the test.py into the function index() in the server.py and modify the .\templates\index.html file(refer to the Flask module in Assignment 1 Part 2 and other online materials) so that when we visit our http server in the browser, we can see the titanic data is displayed in graphic mode in a table.

https://www.tutorialspoint.com/flask/flask_templates.htm

Note 1: You need to do some self-study to be able to modify the server.py and index.html correctly

Note 2: You need to restart the cloudcompute.service in order to see the modification you made on server.py/index.html via

\$ sudo systemctl restart cloudcompute.service

Step 8. Check your new web page with Titanic data:

Relaunch your http server on the EC2 instance after finishing step 7, type in its url in the browser to check the output of titanic data.

Assignment 2

Amazon EC2 RDS

(updated Sept. 2025)

Assignment Deliverables

1. Document the creation of two security groups sgRDS and sgWEB that includes:

1.1 Screenshot of the sgRDS that shows the inbound rules (10 pts)

sg-0ca9608b150bdc9c3 - sgRDS

Actions ▾

Details

Security group name

sgRDS

Security group ID

sg-0ca9608b150bdc9c3

Description

security group for RDS

VPC ID

vp-091ffc5ef10a67c8f

Owner

293578648063

Inbound rules count

1 Permission entry

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing - new

VPC associations - new

Tags

Inbound rules (1)

Manage tags

Edit inbound rules

Search

< 1 >

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0c424add17df5c030	-	MySQL/Aurora	TCP	3306

1.2 Screenshot of the sgWEB that shows the inbound rules (10 pts)

sg-0fa354fcd94b7fa0f - sgWEB

Actions ▾

Details

Security group name

sgWEB

Security group ID

sg-0fa354fcd94b7fa0f

Description

security group for Web access

VPC ID

vp-091ffc5ef10a67c8f

Owner

293578648063

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing - new

VPC associations - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

Search

< 1 >

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-01d0a782921bef2d8	IPv4	HTTP	TCP	80
☐	-	sgr-036523bc775144163	IPv4	SSH	TCP	22

2. Document the creation of RDS that includes:

2.1 Screenshot of the RDS database summary page, “connectivity & security” part that includes Endpoint & Port, Networking, Security and Security Group Rules. (10 pts)

The screenshot shows the Amazon RDS console for an Aurora instance named 'database-cloudcompute'. The left sidebar contains navigation links for Aurora and RDS, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations (0), and Certificate update.

The main content area is divided into two sections: Summary and Connectivity & security.

Summary:

- DB identifier:** database-cloudcompute
- Status:** Available
- Role:** Instance
- Engine:** MySQL Community
- Recommendations:**
- CPU:** 3.22%
- Class:** db.t3.micro
- Current activity:** 0 Connections
- Region & AZ:** us-west-1c

Connectivity & security:

- Endpoint & port:**
 - Endpoint:** database-cloudcompute.crg0eawi4fqw.us-west-1.rds.amazonaws.com
 - Port:** 3306
- Networking:**
 - Availability Zone:** us-west-1c
 - VPC:** vpc-091ffc5ef10a67c8f
 - Subnet group:** default-vpc-091ffc5ef10a67c8f
 - Subnets:** subnet-05dca873c49f84a59, subnet-0e7991bcf1aba410c
 - Network type:** IPv4
- Security:**
 - VPC security groups:** sgRDS (sg-0ca9608b150bdc9c3) - Active
 - Publicly accessible:** No
 - Certificate authority:** rds-ca-rsa2048-g1
 - Certificate authority date:** May 19, 2061, 13:04 (UTC-07:00)
 - DB instance certificate expiration date:** October 06, 2026, 01:48 (UTC-07:00)

3. Document the connection from EC2 to RDS:

3.1 Screenshot of the running results of step 5: (20 pts)

- the output of executing createdB.sql; (5 pts)

```
[mysql> source createdB.sql;
Query OK, 1 row affected (0.03 sec)

Database changed
Query OK, 0 rows affected (0.07 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 1 row affected (0.00 sec)
```

- the output of show databases; (5 pts)

```
[mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| testdb |
+-----+
5 rows in set (0.00 sec)
```

- the output of show tables; (5 pts)

```
[mysql> show tables;
+-----+
| Tables_in_testdb |
+-----+
| titanic |
+-----+
1 row in set (0.01 sec)
```

- the output of select query. (5 pts)

```
[mysql> select * from titanic;
+----+-----+-----+-----+-----+
| id | pclass | name | sex | age |
+----+-----+-----+-----+-----+
| 892 | 3 | Kelly | male | 34.5 |
| 893 | 3 | Wilkes | female | 47 |
| 894 | 2 | Myles | male | 62 |
| 900 | 3 | Abraham | female | 18 |
| 903 | 1 | Jones | male | 46 |
+----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

3.2 Screenshot of the running result of step 6: the output of test.py (10 pts)

```
[ec2-user@ip-172-31-17-121 cloudcompute]$ python3 test.py
uid:892, pclass:3, name:Kelly, sex:male, age:34.5
uid:893, pclass:3, name:Wilkes, sex:female, age:47.0
uid:894, pclass:2, name:Myles, sex:male, age:62.0
uid:900, pclass:3, name:Abraham, sex:female, age:18.0
uid:903, pclass:1, name:Jones, sex:male, age:46.0
```

4. Coding:

4.1 attachment of file server.py in step 7 (10 pts)

```
[ec2-user@ip-172-31-17-121 cloudcompute]$ cat server.py
#!/usr/bin/python3
from flask import Flask, render_template
import pymysql, os, sys

# ---- DB config (use the same values that worked in mysql client) ----
DB_HOST = os.getenv("DB_HOST", "database-cloudcompute.crg0eawi4fqw.us-west-1.rds.amazonaws.com")
DB_USER = os.getenv("DB_USER", "admin")
DB_PASS = os.getenv("DB_PASS", "Parineeta28")
DB_NAME = os.getenv("DB_NAME", "testdb")

app = Flask(__name__)

def get_rows():
    # Use DictCursor so template can use keys like row.uid
    conn = pymysql.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASS,
        database=DB_NAME,
        cursorclass=pymysql.cursors.DictCursor,
        connect_timeout=5,
    )
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT id as uid, pclass, name, sex, age FROM titanic ORDER BY id;")
            return cur.fetchall()
    finally:
        conn.close()

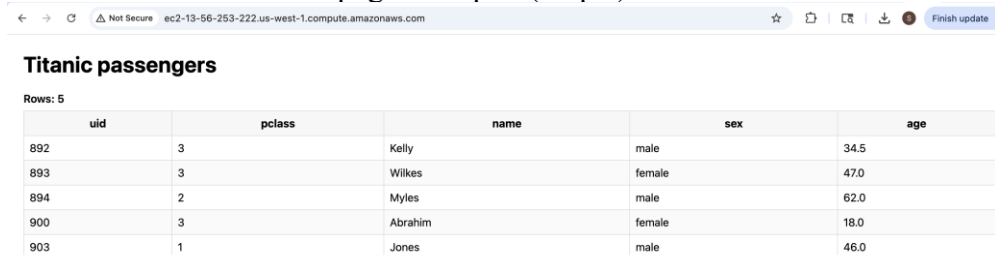
@app.route("/")
def index():
    try:
        rows = get_rows()
        columns = ["uid", "pclass", "name", "sex", "age"]
        return render_template("index.html", rows=rows, columns=columns)
    except Exception as e:
        # Simple error page to help debugging in the browser
        return f"<h1>Error</h1><pre>{e}</pre>", 500

if __name__ == "__main__":
    # For manual testing you can: PORT=8080 python3 server.py
    port = int(os.getenv("PORT", "80"))
    app.run(host="0.0.0.0", port=port)
```

4.2 attachment of file index.html at step 8 (10 pts)


```
[ec2-user@ip-172-31-17-121 cloudcompute]$ cd templates/
[ec2-user@ip-172-31-17-121 templates]$ ls
index.html
[ec2-user@ip-172-31-17-121 templates]$ cat index.html
<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Titanic Data</title>
    <style>
      body { font-family: system-ui, Arial, sans-serif; margin: 24px; }
      table { border-collapse: collapse; width: 100%; }
      th, td { border: 1px solid #ddd; padding: 8px; }
      th { background: #f7f7f7; position: sticky; top: 0; }
      tr:nth-child(even) { background: #fafafa; }
      caption { text-align: left; font-weight: 600; margin-bottom: 8px; }
    </style>
  </head>
  <body>
    <h1>Titanic passengers</h1>
    <table>
      <caption>Rows: {{ rows|length }}</caption>
      <thead>
        <tr>
          {% for col in columns %}
            <th>{{ col }}</th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
          <tr>
            <td>{{ row.uid }}</td>
            <td>{{ row.pclass }}</td>
            <td>{{ row.name }}</td>
            <td>{{ row.sex }}</td>
            <td>{{ row.age }}</td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  </body>
</html>
```

4.3 Screenshot of the webpage at step 9. (10 pts)



uid	pclass	name	sex	age
892	3	Kelly	male	34.5
893	3	Wilkes	female	47.0
894	2	Myles	male	62.0
900	3	Abraham	female	18.0
903	1	Jones	male	46.0

4.4 tree structure of your directory 'cloudcompute' in the EC2 instance (10 pts)

```
[ec2-user@ip-172-31-17-121 cloudcompute]$ tree
.
├── createDB.sql
├── server.py
├── server_1.py
├── templates
│   └── index.html
└── test.py

1 directory, 5 files
```